

Project Specification: Fake News Detection + Explainability

Problem

Build a system to detect fake news articles using text classification, with an **explainability layer** to highlight key words contributing to each prediction.

Dataset

- Fake News Dataset (e.g., Kaggle: *Fake and Real News Dataset*).
 - Contains: headline, text, label (fake/real).
-

Stepwise Plan

Step 1: Problem Understanding & Data Requirements

- Objective: classify articles into **fake** vs **real**.
 - Secondary objective: provide **explainability** (why an article is classified as fake/real).
 - Target metric: Accuracy, F1-score, Precision/Recall.
-

Step 2: Data Collection & Exploration

- Acquire **Fake News Dataset**.
 - Perform exploratory data analysis (EDA):
 - Class distribution (fake vs real).
 - Text length distribution.
 - Common words/phrases.
-

Step 3: Data Preprocessing & Feature Engineering

- Clean text: lowercasing, stopword removal, punctuation handling.
 - Tokenization & lemmatization.
 - Create two sets of features:
 1. **BoW/TF-IDF vectors** → baseline.
 2. **Word embeddings (e.g., GloVe/Word2Vec)** → deep models.
-

Step 4: Baseline Model Development

- Train **Logistic Regression / Naive Bayes** with BoW/TF-IDF.
 - Evaluate with Accuracy, F1-score.
 - Save baseline performance for MLflow tracking.
-

Step 5: Advanced Model Development (Deep Learning)

- Use **CNN** for text classification with word embeddings.
 - Compare CNN with **LSTM** (optional, sequential focus).
 - Optimize hyperparameters.
 - Log all experiments in MLflow.
-

Step 6: Explainability Layer

- Use **LIME/SHAP** to show top words contributing to predictions.
 - Build function to highlight important words in article text.
 - Integrate explainability for both ML and DL models.
-

Step 7: Pipeline Orchestration with Airflow

- Create DAG for:
 1. Data ingestion (new news articles).
 2. Preprocessing.
 3. Model training/retraining.
 4. Evaluation & explainability reporting.
-

Step 8: Experiment Tracking & Versioning with MLflow

- Track:
 - Model type (LogReg, CNN, etc.).
 - Parameters (learning rate, embedding size, etc.).
 - Metrics (accuracy, F1).

- Explainability samples.

Step 9: Deployment with Docker & Kubernetes

- Containerize app with **Docker**.
- Deploy REST API on **Kubernetes cluster**.
 - Endpoint /predict: takes in article text → returns label + explanation.
- Scale with Kubernetes pods for multiple requests.

Step 10: Monitoring & Maintenance

- Log prediction requests and accuracy drift.
- Schedule Airflow retraining if drift detected.
- Monitor explanation quality (ensure model doesn't rely on irrelevant tokens).

Deliverables

1. **EDA Report** (class balance, word distributions, etc.).
2. **Preprocessing Scripts** (text cleaning, feature extraction).
3. **Baseline ML models** (Logistic Regression/Naive Bayes).
4. **CNN/LSTM models** with embeddings.
5. **Explainability module** (LIME/SHAP visualizations).
6. **MLflow experiment tracking logs**.
7. **Airflow DAGs** (ETL + training pipeline).
8. **Dockerized app** with REST API.
9. **Kubernetes deployment manifests**.
10. **Final report + demo notebook** (classification + explainability showcase).